

编译原理实验 PR003

组员：谢翊凡，闫涛，曹建文

Part1: 实验思路

本次实验中，我们做的主要工作分为两部分：首先是对 `main.cpp` 等文件相关内容做一定修改以适配后续实验内容，其次是本次实验的主要工作部分，即 `RISCVCodeGenerator.h` 与 `RISCVCodeGenerator.cpp` 文件的设计与编写。其中 `RISCVCodeGenerator.h` 主要提供需要用到的接口和宏定义，`RISCVCodeGenerator.cpp` 则主要完成目标代码的生成。它完成了一个代码生成器，接收一个 LLVM 模块 (`llvm::Module`)，遍历其中的全局变量、函数、基本块和指令，将其转换为对应的 RISC-V 汇编代码，并输出到指定文件中。以此实现中间代码到目标代码的转换。

关于目标代码生成器部分，我们的主要实现思路如下：

1. 通过 `DEBUG_LOG` 宏控制调试日志的输出，方便开发过程中跟踪代码生成过程。

2. 主生成流程

`generate` 方法是入口函数，负责打开输出文件，调用 `visitModule` 遍历 LLVM 模块，最后关闭文件。

3. 对全局变量进行处理，如：

`processGlobals` 方法遍历模块中的全局变量，为每个全局变量分配存储空间，并根据是否有初始值输出相应的汇编指令。

4. 对函数进行处理，如：

栈帧分配：`visitFunction` 方法会为函数分配栈帧，包括固定区域（保存返回地址和旧帧指针）和局部变量区域。计算栈帧大小时会考虑函数参数、`alloca` 指令和指令结果的空间需求，并将栈帧大小对齐到 16 字节边界。

栈偏移分配：为函数参数、`alloca` 指令和指令结果分配栈偏移，存储在 `stack_offsets` 中。

代码生成：调用 `functionPrologue` 生成函数序言，保存返回地址和旧帧指针，设置新的帧指针；处理函数参数的存储；遍历基本块生成代码；调用 `functionEpilogue` 生成函数尾声，恢复寄存器和栈指针。

5. 对基本块进行处理，如：

`visitBasicBlock` 方法为每个基本块生成标签，并遍历其中的指令，调用 `visitInstruction` 处理。

6. 对指令进行处理，如：

`visitInstruction` 方法根据指令类型进行分发，调用不同的处理函数，如 `visitReturnInst`、`visitBinaryOperator` 等。

7. 辅助函数的设计与实现，如：

`getStackAddress`：获取值的栈地址。

`loadValueToReg`：将值加载到指定寄存器。

`storeRegToValue`：将寄存器的值存储到指定值对应的栈位置。

`loadFloatValueToReg` 和 `storeFloatRegToValue`：处理浮点值的加载和存储。

`emitConstant`：输出常量的汇编表示。

8. 具体指令实现

针对不同的 LLVM 指令，实现了对应的处理函数，如 `visitReturnInst` 处理返回指令，`visitBinaryOperator` 处理二元操作指令等。

Part2: 主要代码实现

首先是之前提到的入口函数：

```
void RISCVCCodeGenerator::generate(llvm::Module* module, const std::string& filename) {
    out.open(filename);
    if (!out.is_open()) {
        std::cerr << "Error: Could not open output file " << filename << std::endl;
        return;
    }
    this->current_module = module;
    visitModule(module);
    out.close();
}
```

图 1: 入口函数及调用流程图示

它作为目标生成的入口函数，主要负责打开输出文件、初始化当前模块信息，调用整体生成过程 `visitModule` 以及关闭文件。若文件打开失败，会输出错误信息并终止操作。

接下来就是从中间代码转换为目标代码的函数过程。转换为目标代码过程中要处理的中间代码“类型”是各自不同的，所以也采用不同函数进行处理。

```
void RISCVCCodeGenerator::processGlobals(llvm::Module* module) {
    out << ".data\n";
    const llvm::DataLayout& dl = module->getDataLayout(); // 获取DataLayout
    for (const llvm::GlobalVariable& global : module->globals()) {
        uint64_t size = dl.getTypeAllocSize(global.getValueType());
        out << " .type " << global.getName().str() << ", @object\n";
        out << " .size " << global.getName().str() << ", " << size << "\n";
        out << global.getName().str() << ":\n";
        if (global.hasInitializer()) {
            emitConstant(out, global.getInitializer()); // **使用新的辅助函数**
        } else {
            uint64_t size = dl.getTypeAllocSize(global.getValueType());
            out << " .zero " << size << "\n";
        }
    }
    out << "\n";
}
```

图 2: 入口函数及调用流程图示

上述代码主要负责对全局变量的处理，其核心逻辑是遍历 LLVM 模块中的所有全局变量，为每个全局变量生成相应的汇编代码。具体步骤如下：

1. 声明进入数据段。
2. 遍历所有全局变量，为每个变量声明类型和大小。
3. 若变量有初始值，调用 `emitConstant` 方法输出初始值；若没有，则用 `.zero` 指令分配零初始化空间。
4. 最后输出一个空行分隔代码。

```
void RISCVCodeGenerator::loadAddressOfValueToReg(llvm::Value* value, const std::string& reg) {
    LOG_ADDR(value, reg);
    if (auto* G = llvm::dyn_cast<llvm::GlobalValue>(value)) {
        out << " la " << reg << ", " << G->getName().str() << "\n";
    } else if (stack_offsets.count(value)) {
        // For a value on the stack, its address is s0 + offset
        out << " addi " << reg << ", s0, " << stack_offsets[value] << "\n";
    } else {
        out << " # ERROR: Cannot get address of this value\n";
    }
}
```

图 3: 入口函数及调用流程图示

这段代码主要定义了 `loadAddressOfValueToReg` 方法，主要功能是将指定值的地址加载到指定的寄存器中。它会根据值的类型（全局变量或栈上值）生成不同的指令。其实现逻辑是根据输入值的类型，生成相应的汇编指令来获取该值的地址并加载到指定寄存器。若值为全局变量，使用 `la` 指令加载其地址；若值在栈上，使用 `addi` 指令计算并加载其地址；若值不属于以上两种情况，则输出错误信息。

这个函数在文件中主要起到辅助作用，是我们设计的“辅助函数”，用于其他函数具体的处理中，减少代码冗余。同样作为辅助函数存在的函数还有：用于获取值的栈地址的 `getStackAddress`，用于将寄存器的值存储到指定值对应的栈位置的 `storeRegToValue` 等等。

```
void RISCVCodeGenerator::loadFloatValueToReg(llvm::Value* value, const std::string& reg) {
    if (auto* CF = llvm::dyn_cast<llvm::ConstantFP>(value)) {
        std::string temp_reg = "t3";
        llvm::APInt int_val = CF->getValueAPF().bitcastToAPInt();
        long long val = int_val.getZExtValue();

        // **修复: 使用 lui 和 addi 加载32位立即数**
        unsigned int high20 = (val >> 12);
        unsigned int low12 = val & 0xFFF;
        if (low12 & 0x800) high20++;

        out << " lui " << temp_reg << ", " << high20 << "\n";
        out << " addi " << temp_reg << ", " << temp_reg << ", " << (short)low12 << "\n";

        std::string temp_slot = "-32(s0)";
        out << " sw " << temp_reg << ", " << temp_slot << "\n";
        out << " flw " << reg << ", " << temp_slot << "\n";
    } else {
        out << " flw " << reg << ", " << getStackAddress(value) << "\n";
    }
}
```

图 4: 入口函数及调用流程图示

再例如 `loadFloatValueToReg`。其主要功能是将一个浮点值加载到指定的 RISC - V 浮点寄存器中。根据输入值的类型，采用不同的加载策略。若输入值是浮点常量，先将其转换为整数表示，再利用 `lui` 和 `addi` 指令将 32 位立即数加载到临时整数寄存器，接着把该值存入栈槽，最后从栈槽加载浮点值到指定的浮点寄存器。若输入值不是浮点常量，直接从栈或全局变量中加载浮点值到指定的浮点寄存器。这样的辅助函数还有很多。

虽然这些函数并不直接产生生成的目标代码，但其作用主要是辅助其他函数完成目标代码的生成。通过辅助函数的设置可以极大降低代码的重复及冗余，提升效率。

```
void RISCVCodeGenerator::visitModule(llvm::Module* module) {
    processGlobals(module);

    out << ".text\n";

    // If there is a main function, declare it as global
    if (module->getFunction("main")) {
        out << ".globl main\n";
        out << ".type main, @function\n";
    }

    for (llvm::Function& F : *module) {
        if (!F.isDeclaration()) {
            visitFunction(F);
        }
    }
}
```

图 5: 入口函数及调用流程图示

这段函数遍历 LLVM 模块，将全局变量和函数转变为目标代码。它首先调用 `processGlobals` 方法处理全局变量，将其放入数据段，然后切换到文本段，准备生成可执行代码。若模块中存在 `main` 函数，将其声明为全局函数；遍历模块中的所有函数，对非声明的函数调用 `visitFunction` 方法生成汇编代码。这个函数是相对上层的控制函数，通过调用实现的 `processGlobals` 函数的等完成工作。下面的 `visitFunction` 也是同理，自身不直接实现具体的汇编代码生成逻辑，而是通过调用其他函数来完成整个代码生成流程。

```

void RISCVCGenerator::visitFunction(llvm::Function& F) {
    LOG_FUNC(F);
    out << "\n" << F.getName().str() << ":\n";
    current_function = &F;
    epilogue_label = "." + F.getName().str() + "_epilogue";
    stack_offsets.clear();

    // --- Pass 1: Allocate stack space ---

    const llvm::DataLayout& dl = F.getParent()->getDataLayout();
    auto getAlignedSize = [&](llvm::Type* type) {
        uint64_t size = dl.getTypeAllocSize(type);
        // 向上对齐到4字节
        return (size + 3) & ~3;
    };

    int size_for_locals = 0; // Total space for locals, temps, and arg backups

    // Space for argument backups on the stack
    for (auto& arg : F.args()) {
        size_for_locals += getAlignedSize(arg.getType());
    }
}

```

图 6: 入口函数及调用流程图示

上面是节选的部分 `visitFunction` 代码，它的主要功能是将 LLVM 函数转换为对应的 RISC-V 汇编代码。它的工作过程大致可以分为三个部分：

第一部分是初始化部分，在工作开始前先做一些初始化操作，以使得后面的代码可以正常运行。

第二部分是从 ``const llvm::DataLayout& dl = F.getParent()->getDataLayout();' 开始的栈空间的分配部分。主要完成的任务有：

- 获取 LLVM 模块中的数据布局信息，用于计算类型的大小；
- 定义一个 lambda 函数 `getAlignedSize`，将类型大小向上对齐到 4 字节；
- 计算局部变量、临时变量和参数等分配所需的栈空间；
- 计算固定区域的大小（用于保存返回地址、旧帧指针和调用者保存寄存器）；
- 计算总的栈帧大小，并将其向上对齐到 16 字节边界等。

第三部分就是目标代码的生成。上面说过 `visitFunction` 是相对上层的控制函数，主要体现在这里：

1. 调用 `functionPrologue` 方法生成函数序言代码，用于保存寄存器和分配栈空间；
2. 将函数参数从寄存器（如 `a0`、`fa0` 等）存储到对应的栈槽中；
3. 遍历函数中的所有基本块，调用 `visitBasicBlock` 方法生成每个基本块的汇编代码；
4. 输出函数尾部标号，调用 `functionEpilogue` 方法生成函数尾声代码，用于恢复寄存器和释放栈空间。

也就是调用不同函数实现相应操作。以 `functionPrologue` 为例：

```

void RISCVCodeGenerator::functionPrologue(llvm::Function& F) {
    // s0 is used as the frame pointer (fp)
    out << " # Function Prologue for " << F.getName().str() << "\n";
    out << " addi sp, sp, -" << current_stack_size << "\n"; // Allocate stack frame
    out << " sw ra, " << (current_stack_size - 4) << "(sp)\n"; // Save return address
    out << " sw s0, " << (current_stack_size - 8) << "(sp)\n"; // Save old frame pointer
    out << " addi s0, sp, " << current_stack_size << "\n"; // Set new frame pointer
}

```

图 7: 入口函数及调用流程图示

这段函数的主要功能是为 RISC - V 汇编代码生成函数序言部分，用于进行栈帧的分配、保存关键寄存器等操作，为函数的执行做准备。主要步骤分为分配栈空间，保存寄存器，设置新栈指针等。与提到的其他几个函数一样被调用以完成功能。

```

void RISCVCodeGenerator::visitBasicBlock(llvm::BasicBlock& BB) {
    LOG_BB(BB);
    // Basic block names from LLVM IR might contain illegal characters, so sanitize them
    out << "." << BB.getParent()->getName().str() << "_" << BB.getName().str() << ":\n";
    for (llvm::Instruction& I : BB) {
        visitInstruction(I);
    }
}

```

图 8: 入口函数及调用流程图示

VisitBasicBlock 是对基本块进行处理，遍历其中所有指令，调用 visitInstruction 分别对每条指令进行处理。而 visitInstruction 的具体实现如下：

```

void RISCVCodeGenerator::visitInstruction(llvm::Instruction& I) {
    LOG_INSTRUCTION(I);
    // Print the instruction as a comment for easier debugging
    std::string instStr;
    llvm::raw_string_ostream rso(instStr);
    I.print(rso);
    out << "  # " << instStr << "\n";

    // Dispatch using llvm::isa and llvm::dyn_cast
    if (auto* inst = llvm::dyn_cast<llvm::ReturnInst>(&I)) {
        visitReturnInst(*inst);
    } else if (auto* inst = llvm::dyn_cast<llvm::BinaryOperator>(&I)) {
        visitBinaryOperator(*inst);
    } else if (auto* inst = llvm::dyn_cast<llvm::AllocaInst>(&I)) {
        // Alloca was already handled in the first pass, do nothing here
    } else if (auto* inst = llvm::dyn_cast<llvm::LoadInst>(&I)) {
        visitLoadInst(*inst);
    } else if (auto* inst = llvm::dyn_cast<llvm::StoreInst>(&I)) {
        visitStoreInst(*inst);
    } else if (auto* inst = llvm::dyn_cast<llvm::BranchInst>(&I)) {
        visitBranchInst(*inst);
    } else if (auto* inst = llvm::dyn_cast<llvm::CallInst>(&I)) {
        visitCallInst(*inst);
    } else if (auto* inst = llvm::dyn_cast<llvm::ICmpInst>(&I)) {
        visitICmpInst(*inst);
    } else if (auto* inst = llvm::dyn_cast<llvm::GetElementPtrInst>(&I)) {
        visitGetElementPtrInst(*inst);
    } else if (auto* inst = llvm::dyn_cast<llvm::PHINode>(&I)) {
        visitPHINode(*inst);
    } else if (auto* inst = llvm::dyn_cast<llvm::CastInst>(&I)) {
        visitCastInst(*inst);
    } else if (auto* inst = llvm::dyn_cast<llvm::FCmpInst>(&I)) {
        visitFCmpInst(*inst);
    }
}

```

图 9: 入口函数及调用流程图示

`visitInstruction` 方法作为指令分派器，通过 `llvm::dyn_cast` 对 LLVM 指令类型进行判断，将不同类型的指令分派到对应的处理函数，从而将 LLVM 中间表示转换为 RISC-V 汇编代码。若遇到未处理的指令类型，会在汇编代码中添加警告注释。`visitReturnInst` 等在 if 判断条件下的函数，都是被调用的，用于处理各种指令的函数。以注释的方式直观表现调用的各函数功能：

```

// 若为返回指令，调用 visitReturnInst 处理
if (auto* inst = llvm::dyn_cast<llvm::ReturnInst>(&I)) {
    visitReturnInst(*inst);
}

// 若为二元操作指令，调用 visitBinaryOperator 处理
else if (auto* inst = llvm::dyn_cast<llvm::BinaryOperator>(&I)) {
    visitBinaryOperator(*inst);
}

// Alloca 指令在第一阶段已处理，此处不做操作
else if (auto* inst = llvm::dyn_cast<llvm::AllocaInst>(&I)) {
}

// 若为加载指令，调用 visitLoadInst 处理
else if (auto* inst = llvm::dyn_cast<llvm::LoadInst>(&I)) {
    visitLoadInst(*inst);
}

// 若为存储指令，调用 visitStoreInst 处理
else if (auto* inst = llvm::dyn_cast<llvm::StoreInst>(&I)) {
}

```

```

    visitStoreInst(*inst);
}
// 若为分支指令，调用 visitBranchInst 处理
else if (auto* inst = llvm::dyn_cast<llvm::BranchInst>(&I)) {
    visitBranchInst(*inst);
}
// 若为函数调用指令，调用 visitCallInst 处理
else if (auto* inst = llvm::dyn_cast<llvm::CallInst>(&I)) {
    visitCallInst(*inst);
}
// 若为整数比较指令，调用 visitCmpInst 处理
else if (auto* inst = llvm::dyn_cast<llvm::ICmpInst>(&I)) {
    visitICmpInst(*inst);
}
// 若为获取元素指针指令，调用 visitGetElementPtrInst 处理
else if (auto* inst = llvm::dyn_cast<llvm::GetElementPtrInst>(&I)) {
    visitGetElementPtrInst(*inst);
}
// 若为 PHI 节点指令，调用 visitPHINode 处理
else if (auto* inst = llvm::dyn_cast<llvm::PHINode>(&I)) {
    visitPHINode(*inst);
}
// 若为类型转换指令，调用 visitCastInst 处理
else if (auto* inst = llvm::dyn_cast<llvm::CastInst>(&I)) {
    visitCastInst(*inst);
}
// 若为浮点比较指令，调用 visitFCmpInst 处理
else if (auto* inst = llvm::dyn_cast<llvm::FCmpInst>(&I)) {
    visitFCmpInst(*inst);
}
// 若为一元操作指令，假设此处为浮点取负
else if (auto* inst = llvm::dyn_cast<llvm::UnaryOperator>(&I)) {
    if (inst->getOpcode() == llvm::Instruction::FNeg) {
        // 加载操作数值到寄存器 f0
        loadFloatValueToReg(inst->getOperand(0), "ft0");
        // 生成负号汇编指令
        out << "fneg.s ft1, ft0\n";
        // 将结果存储到对应的值位置
        storeFloatRegToValue("ft1", inst);
    }
}
// 若指令类型未被处理，输出警告信息
else {
    // TODO: warning
}

```

以下不妨以其中之一为例：


```

void RISCVCodeGenerator::visitReturnInst(llvm::ReturnInst& I) {
    if (llvm::Value* retVal = I.getReturnValue()) {
        if (retVal->getType()->isFloatTy()) {
            loadFloatValueToReg(retVal, "fa0"); // **修复：浮点返回值用 fa0**
        } else {
            loadValueToReg(retVal, "a0");
        }
    }
    out << "    j    " << epilogue_label << "\n";
}

```

图 10: 入口函数及调用流程图示

其主要功能是将 LLVM 的 ReturnInst 指令转换为对应的 RISC - V 汇编代码，处理函数返回值并跳转到函数尾声部分。

若返回指令带有返回值，依据返回值类型将其加载到对应的寄存器。浮点类型返回值加载到 fa0 寄存器，非浮点类型返回值加载到 a0 寄存器；

无论是否有返回值，都使用 j 指令跳转到函数尾声标签处，进而执行函数尾声代码（如恢复寄存器、释放栈空间等）。

与此类似的涉及到的其它函数也是同理，分别完成各自需要处理的部分，在被调用的时候完成工作。

Part3: 总结

本次实现实现了从中间代码到目标代码的转换。我们选择直接对输入的中间代码处理再目标代码以输出。对于不同中间代码做出不同处理的部分，我们采取了“条件判断语句 + 细化类别的方法实现”的策略，同时对于可能多次涉及的、非直接产生目标代码结果的辅助性操作，我们设计了一系列辅助函数，以减少代码冗余或代码重复。对于目标代码的优化部分，目前尚未提升空间。上述设计可以完成目标代码的生成，但如果更进一步，目标代码的优化将会是一个重要的点。